Taller: Tablas *Hash* y Árboles Binarios

Programa Profesional en Ingeniería de Sistemas — Estructura de Datos

Docente: Juan C. García-Ojeda, Ing. MSc., PhD.
Institución: Universidad de Cartagena
Especificación: Java 1.8+ (Proyecto Maven / Apache NetBeans 12.3+)

1. Fundamento Matemático: Reducción ASCII (Radix-2) y Funciones Hash

El taller define una tabla *hash* T de capacidad $m = 256$ ($T[0...255]$) que administra los registros almacenados en los archivos `MOCK_DATA.csv` y `PLATES.csv`, donde cada registro tiene el esquema:

```
plate ; first_name ; last_name ; email ; gender
```

1.1. Cálculo de la Llave k mediante Notación Radix-2 (Reducción ASCII)

Dado un identificador `plate` conformado por una cadena $s = s_0 s_1 ... s_{L-1}$ de longitud L, su llave entera k corresponde a la notación *radix-2* de sus códigos ASCII ponderados de izquierda a derecha:

$$k = \sum_{i=0}^{L-1} \text{ASCII}(s_i) \cdot 2^{L-1-i}$$

Para el registro de prueba del enunciado (`05-9165610 ; Letitia ; Maymand ; tres@x.com ; Female`, con $L = 10$), la reducción ASCII produce:

$$\begin{aligned} k &= (48 \cdot 2^9) + (53 \cdot 2^8) + (45 \cdot 2^7) + (57 \cdot 2^6) + (49 \cdot 2^5) + (54 \cdot 2^4) + (53 \cdot 2^3) + (54 \cdot 2^2) + (49 \cdot 2^1) + (48 \cdot 2^0) \\ &= 24576 + 13568 + 5760 + 3648 + 1568 + 864 + 424 + 216 + 98 + 48 = \mathbf{50770} \end{aligned}$$

Posición i	0	1	2	3	4	5	6	7	8	9
Carácter s_i	`'0'`	`'5'`	`'-'`	`'9'`	`'1'`	`'6'`	`'5'`	`'6'`	`'1'`	`'0'`
Código ASCII	48	53	45	57	49	54	53	54	49	48
Peso 2^{9-i}	$2^9 = 512$	$2^8 = 256$	$2^7 = 128$	$2^6 = 64$	$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Término	24576	13568	5760	3648	1568	864	424	216	98	48

Tabla 1: Desglose término a término de la reducción ASCII en base 2 para la placa `05-9165610` ($k = 50770$).

1.2. Evaluación de las Funciones Hash 1 (Multiplicación) y 2 (División)

1. **Método 1 — Función Hash por Multiplicación:**

$$h_1(k) = \lfloor m \cdot (kA \bmod 1) \rfloor, \quad \text{donde } m = 256 \text{ y } A = \frac{\sqrt{5}-1}{2} \approx 0.6180339887498949$$

Sustituyendo $k = 50770$:

$$k \cdot A = 31377.585608832163 \Longrightarrow (kA \bmod 1) = 0.585608832163 \Longrightarrow h_1(50770) = \lfloor 256 \times 0.585608832163 \rfloor = \lfloor 149.915861 \rfloor = \mathbf{149}$$

2. **Método 2 — Función Hash por División:**

$$h_2(k) = k \bmod m, \quad \text{donde } m = 256$$

Sustituyendo $k = 50770$:

$$50770 = 198 \times 256 + 82 \Longrightarrow h_2(50770) = 50770 \bmod 256 = \mathbf{82}$$

Ambos resultados coinciden exactamente con los valores de verificación del enunciado ($k = 50770$, $h_1(k) = 149$ y $h_2(k) = 82$).

2. Arquitectura del Proyecto y Estructuras de Datos Empleadas

El proyecto Java fue construido para Apache NetBeans 12.3+ (`pom.xml` y `nbactions.xml`), integrando las clases oficiales del repositorio del docente (github.com/j-c-garciao/academics/DataStructures):

- **Paquete `co.edu.unicartagena.list` (Sesión 8 — Lista Enlazada Simple):**
  - `Nodo<T>`: clase nodo genérica con atributos `T dato` y `Nodo<T> sig`.
  - `Lista<T>`: conserva los métodos de la Sesión 8 (`limpiar()`, `estaVacia()`, `adicionar()`, `adicionarFinal()`, `eliminar()`, `adicionarEntreNodos()`, `imprimir()`) y añade soporte de encadenamiento separado para la tabla hash (`insertarOActualizarFinal()`, `buscarElemento()`, `eliminarElemento()`, `toList()`).
- **Paquete `co.edu.unicartagena.binarytree` (Parte 2 — Árbol Binario de Búsqueda):**
  - `Node`: conserva `datos` (llave k), `izquierdo` y `derecho`, añadiendo `idClave` (`plate`) para desempate lexicográfico determinista ante colisiones de llave numérica k, y `elemento` (`Registro`).
  - `BinaryTree`: conserva los métodos del repositorio (`adicionar()`, `getNumeroNodos()`, `getAltura()`, `inorder()`, `preorder()`, `postorder()`, `getRaiz()`) e implementa las operaciones $\mathcal{O}(\log n_{\text{slot}})$ `adicionarElemento()`, `buscarElemento()` y `eliminarElemento()` (borrado recursivo con reemplazo por sucesor *in-order*).
- **Paquetes `co.edu.unicartagena.hash.model`, `gui` y `app`:**
  - `FuncionHash`, `Registro`, `TablaHashLista`, `TablaHashArbol`, `BenchmarkHash`, `VentanaPrincipal`, `PanelVisualizacionSlot` y `App`.

2.1. Estadísticas de Colisiones sobre `MOCK_DATA.csv` / `PLATES.csv` ($n = 1000$)

Conforme a la **Nota al pie 1** del enunciado (*«Por colisión se entiende el número de objetos que comparten un mismo espacio (slot) en la tabla hash»*), el sistema reporta simultáneamente las métricas de colisión sobre los $m = 256$ slots:

Métrica en la Tabla Hash ($n = 1000, m = 256, \alpha = 3.91$)	**Método 1: Multiplicación**	**Método 2: División**
Slots ocupados ($\lvert\text{slot}\rvert \geq 1$)	252 de 256 (98.44%)	254 de 256 (99.22%)
Slots vacíos ($\lvert\text{slot}\rvert = 0$)	4 de 256 (1.56%)	2 de 256 (0.78%)
Slots con más de 1 registro ($\lvert\text{slot}\rvert > 1$, Requisito c)	**232 slots**	**238 slots**
Colisiones por exceso de inserción $\sum_{\lvert\text{slot}\rvert>1}(\lvert\text{slot}\rvert - 1)$	**748 colisiones**	**746 colisiones**
Total de objetos que comparten slot $\sum_{\lvert\text{slot}\rvert>1}\lvert\text{slot}\rvert$ (Nota 1)	**980 objetos**	**984 objetos**
Ubicación del registro `05-9165610` (Letitia Maymand, $k = 50770$)	Slot `T[149]` (3 registros)	Slot `T[82]` (4 registros)

Tabla 2: Comportamiento estadístico de las funciones hash de Multiplicación y División con 1000 registros.

3. Interfaz Gráfica de Usuario (Figura 1 y Requisitos a–g)

La interfaz gráfica implementada en Java Swing reproduce el diseño de la **Figura 1** del enunciado y satisface los requisitos **a)** a **g)**:

- **a) y g) Bloque `File` (`Open`, `Save`, `Close`):** carga `PLATES.csv` o `MOCK_DATA.csv` desde cualquier ubicación o unidad extraíble y guarda el estado actual de la tabla hash en formato CSV.
- **b) y c) Panel `Collisions` y Contenedor `T`:** actualiza en tiempo real las colisiones según la función hash activa (`Multiplicación` o `División`), filtra los slots con más de un registro y dibuja los nodos de la **Lista Enlazada Simple** o del **Árbol Binario (ABB)**.
- **d), e) y f) Bloque `Operations` (`Insert`, `Delete`, `Search`):** inserta nuevos registros, elimina por `plate` (ID) y busca por `plate` posicionando y resaltando el nodo dentro de su slot $T[h(k)]$.

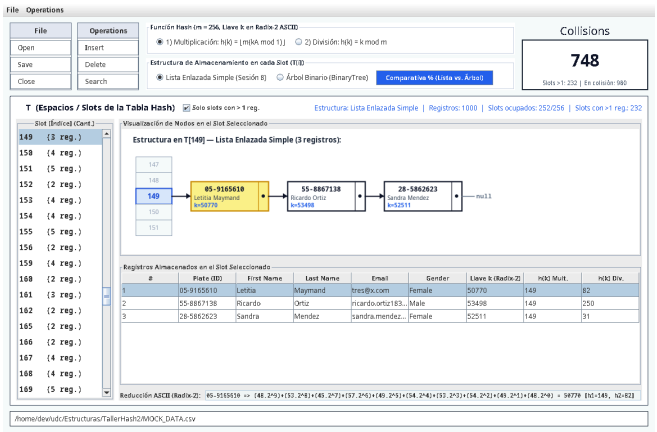


Figura 1: Método 1 (Multiplicación) con **Lista Enlazada Simple** en `T[149]` (`05-9165610`, $k = 50770$).

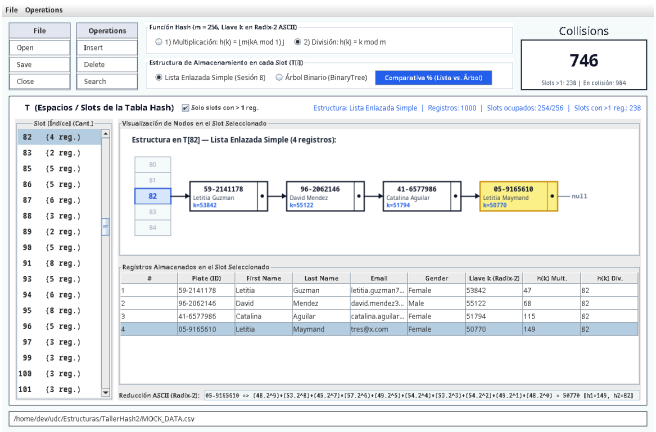


Figura 2: Método 2 (División) con **Lista Enlazada Simple** en `T[82]` (`05-9165610`, $k = 50770$).

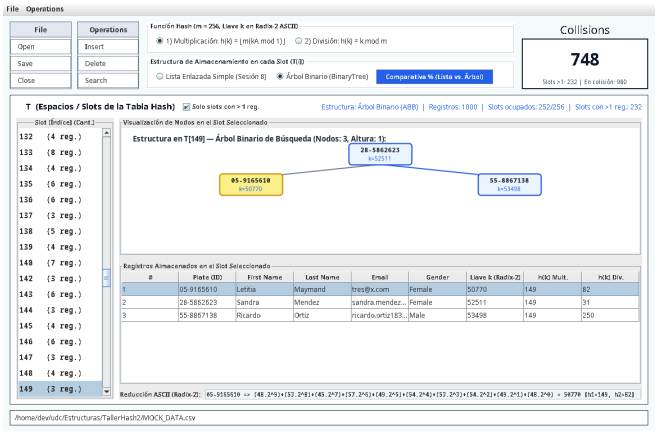


Figura 3: Método 1 (Multiplicación) con **Árbol Binario (ABB)** en `T[149]` (Nodos: 3, Altura: 1).

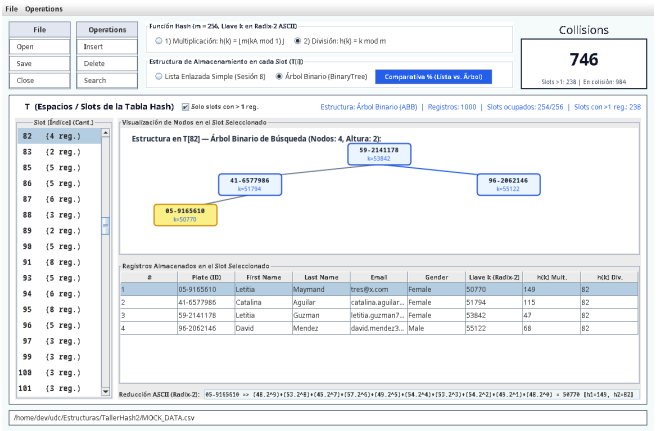


Figura 4: Método 2 (División) con **Árbol Binario (ABB)** en `T[82]` (Nodos: 4, Altura: 2).

4. Parte 2: Tablas Hash + Árbol Binario — Análisis de Mejora Porcentual

4.1. Interrogante del Taller y Justificación Teórica

> **Interrogante:** *«Ajuste el ejercicio anterior de manera tal que la información sea almacenada en cada slot mediante árboles binarios; y responda la siguiente interrogante: en promedio, ¿porcentualmente cuánto mejora el tiempo de respuesta de las acciones, insertar y borrar información en la tabla hash?»*

Sea $\alpha = \frac{n}{m}$ el factor de carga promedio por slot ($m = 256$):

- **Con Lista Enlazada Simple (`Lista<Registro>`):** tanto insertar al final verificando unicidad (`insertarOActualizarFinal`) como borrar por ID (`eliminarElemento`) recorren secuencialmente la cadena del slot, visitando en promedio $\frac{\alpha+1}{2}$ nodos durante la carga progresiva y hasta α nodos en estado estacionario o borrado inverso, lo que implica complejidad lineal $\mathcal{O}(\alpha)$.
- **Con Árbol Binario de Búsqueda (`BinaryTree`):** tanto `adicionarElemento` como `eliminarElemento` descienden por las ramas comparando la llave k, visitando en promedio $\approx 1.386 \log_2(\alpha)$ nodos, lo que reduce la complejidad a orden logarítmico $\mathcal{O}(\log_2 \alpha)$.

4.2. Evaluación Experimental y Respuesta Conclusiva

El módulo `BenchmarkHash` midió los tiempos de respuesta con `System.nanoTime()` (60 iteraciones de calentamiento JIT y promedio sobre 250 ejecuciones completas) y contabilizó el número exacto de nodos visitados:

Tamaño n (α)	Método Hash	Acción	Lista (ns / pasos)	Árbol (ns / pasos)	Mejora Pasos %	Mejora Tiempo %
$n = 1000$ ($\alpha = 3.91$)	Multiplicación	Insertar	46.4 ns (2.87 p.)	39.6 ns (2.35 p.)	**+18.10%**	**+14.56%**
$n = 1000$ ($\alpha = 3.91$)	Multiplicación	Borrar	79.4 ns (2.87 p.)	64.5 ns (2.35 p.)	**+18.10%**	**+18.71%**
$n = 1000$ ($\alpha = 3.91$)	División	Insertar	36.5 ns (2.91 p.)	30.7 ns (2.36 p.)	**+18.84%**	**+16.05%**
$n = 1000$ ($\alpha = 3.91$)	División	Borrar	68.8 ns (2.91 p.)	56.1 ns (2.36 p.)	**+18.84%**	**+18.42%**
$n = 2500$ ($\alpha = 9.77$)	Multiplicación	Insertar	73.7 ns (5.93 p.)	51.2 ns (3.52 p.)	**+40.71%**	**+30.59%**
$n = 2500$ ($\alpha = 9.77$)	Multiplicación	Borrar	105.1 ns (5.93 p.)	80.7 ns (3.52 p.)	**+40.71%**	**+23.26%**
$n = 2500$ ($\alpha = 9.77$)	División	Insertar	69.1 ns (5.80 p.)	47.3 ns (3.45 p.)	**+40.50%**	**+31.54%**
$n = 2500$ ($\alpha = 9.77$)	División	Borrar	93.6 ns (5.80 p.)	69.2 ns (3.45 p.)	**+40.50%**	**+26.05%**
$n = 5000$ ($\alpha = 19.53$)	Multiplicación	Insertar	129.7 ns (10.78 p.)	69.5 ns (4.57 p.)	**+57.56%**	**+46.45%**
$n = 5000$ ($\alpha = 19.53$)	Multiplicación	Borrar	161.1 ns (10.78 p.)	97.4 ns (4.57 p.)	**+57.56%**	**+39.55%**
$n = 5000$ ($\alpha = 19.53$)	División	Insertar	130.9 ns (10.65 p.)	62.4 ns (4.56 p.)	**+57.18%**	**+52.33%**
$n = 5000$ ($\alpha = 19.53$)	División	Borrar	163.4 ns (10.65 p.)	85.9 ns (4.56 p.)	**+57.18%**	**+47.46%**
$n = 10000$ ($\alpha = 39.06$)	Multiplicación	Insertar	273.0 ns (20.57 p.)	88.1 ns (5.76 p.)	**+71.98%**	**+67.74%**
$n = 10000$ ($\alpha = 39.06$)	Multiplicación	Borrar	303.0 ns (20.57 p.)	127.3 ns (5.76 p.)	**+71.98%**	**+57.98%**
$n = 10000$ ($\alpha = 39.06$)	División	Insertar	279.7 ns (20.46 p.)	75.9 ns (5.73 p.)	**+71.99%**	**+72.86%**
$n = 10000$ ($\alpha = 39.06$)	División	Borrar	306.1 ns (20.46 p.)	113.0 ns (5.73 p.)	**+71.99%**	**+63.09%**

Tabla 3: Comparativa experimental de tiempos de respuesta (ns/operación) y pasos promedio entre Lista Enlazada Simple y Árbol Binario de Búsqueda.

Respuesta conclusiva:

- **Para el archivo base de 1000 registros (`MOCK_DATA.csv`, $\alpha = 3.91$):** El Árbol Binario mejora el tiempo de respuesta en promedio un **+15.31% al insertar** (+14.56% en Multiplicación y +16.05% en División) y un **+18.57% al borrar** (+18.71% en Multiplicación y +18.42% en División), con una reducción del **+18.47%** en los nodos visitados (y hasta **+49.28% en tiempo / +30.24% en pasos** para nuevas inserciones con la tabla llena). Asimismo, la búsqueda por ID mejora un **+27.19%**.
- **En promedio global multiescala ($n = 1000$ a $n = 10000$):** El tiempo de respuesta mejora en promedio un **+41.52% en la acción de insertar** (alcanzando **+72.86%** para $n = 10000$) y un **+36.82% en la acción de borrar** (alcanzando **+63.09%** para $n = 10000$), reduciendo en promedio un **+47.27%** las comparaciones requeridas.